Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Sangam: Your City Your Experience

Team Name: *Team #T059*	*Horizen*
Team member 1 (Team Lead)	*Dhasmana, Srishti: 2510010598* *dhasmanasrishti715@gmail.com* 
Team member 2	*Bijalwan, Ishita: 2510380152* *ishitabijalwan.ib@gmail.com* 
Team member 3	*Rana, Nandini: 2510370024* *nr.nandini21@gmail.com* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

Sangam is a tourism web application where a user can explore a city on an interactive map, mark places like tourist attractions, hotels and restaurants, and make a personal travel plan. The user searches for a place, adds it to a trip and gets a route that joins all the chosen stops. The frontend is made with HTML, CSS, JavaScript and Leaflet.js. The main logic is written in C++, because the course wants us to use data structures inside a real application and not only in small practice programs.

The main aim is to show how common data structures solve everyday problems. A Trie gives prefix based search for the autocomplete box, and a hash table finds a place quickly when its name is known. Places and roads are stored as a weighted graph using an adjacency list. Dijkstra's algorithm with a priority queue finds the shortest path, and A* is planned for larger graphs. The itinerary part orders the chosen stops so that the total distance stays small.

Place data is kept in vectors while the program runs and is saved in an SQLite database. Linked lists are used for grouping places by category and for managing the order of stops in an itinerary.

Every member of the team has two modules, and each member has both application work and C++ data structure work. At the end a user should be able to search a place, build an itinerary, see the route on the map and open the saved trip later.

Updated Project Approach and Architecture

The system has three layers: the frontend, the C++ backend and the storage layer.

The frontend is a web page written in HTML, CSS and JavaScript. It uses Leaflet.js with OpenStreetMap tiles to show the map, the markers and the final route. The search box sends the typed text to the backend after a short delay, so the server is not called on every key press.

The backend is in C++ and holds all the data structure logic. When it starts, it loads the places into memory. A Trie is built from the place names for prefix search, and a hash table maps each place name to its record. The places also become the nodes of a weighted graph stored as an adjacency list, and the roads are the edges with distance as the weight. For routing we use Dijkstra's algorithm with a priority queue, and we plan to add A* with a straight line distance heuristic and compare the two. Place records are kept in vectors, and linked lists are used for category groups and for the ordered stops of an itinerary.

For storage we use SQLite with tables for places, categories, itineraries and the stops of each itinerary, so nothing is lost when the program is closed.

The frontend and the backend talk through HTTP requests with JSON, for example one request for search suggestions and one for a route between selected places. The libraries used so far are Leaflet.js on the client side and SQLite3 on the server side.

Tasks Completed

Task Completed	Team Member
Interactive map frontend. Built the main page with HTML, CSS and JavaScript and added the Leaflet.js map. Users can pan, zoom, click markers and see place details in a popup, and the layout works on laptop and phone screens.	Srishti Dhasmana
Place search and autocomplete. Wrote the Trie in C++ for prefix based search and a hash table for fast place lookup, then connected both to the search box so suggestions show up while the user types.	Srishti Dhasmana
Graph construction. Created the graph using an adjacency list, with places as nodes and roads as weighted edges, along with functions to add nodes, add edges and read the neighbours of a node.	Ishita Bijalwan
Route and itinerary optimisation. Implemented Dijkstra's algorithm with a priority queue for the shortest path, and used it to order the stops of an itinerary so the total distance is kept small.	Ishita Bijalwan
Database schema and persistence. Designed the SQLite tables for places, categories and itineraries, and wrote the C++ code that creates the database, stores place data and reads it back.	Nandini Rana
Place and itinerary storage. Stored place data in vectors and used linked lists for grouping places by category and for managing the order of stops inside an itinerary.	Nandini Rana

Challenges/Roadblocks

Road data for the graph. Real road data is large and messy, so building a graph from it is harder than it looks. For now we use a smaller set of places and distances, which is enough to test the algorithms. Later we will try a bigger dataset.

Visiting many stops. Finding the best order for many stops is much harder than a single shortest path, and running Dijkstra between every pair is costly. We chose a simple greedy method that always goes to the nearest unvisited stop, and we accept that the result may not be perfect.

Connecting C++ with the browser. The frontend is JavaScript and the logic is C++, so both sides must agree on the request and response format. We faced some JSON formatting and cross origin problems. We plan to fix a clear list of API endpoints and test each one alone before joining the pages.

Using SQLite from C++. The SQLite C interface needs careful handling of statements and error codes, and our first attempts failed on small mistakes. We solved this by wrapping the database calls in a few helper functions.

Team coordination. Three people work on different modules together, so merge conflicts and mismatched function names are possible. We use separate branches, agree on header files early and meet regularly to compare progress.

Tasks Pending

Task Pending	Description	Team Member
A* Algorithm and Performance Comparison	Complete the A* shortest-path implementation using an appropriate distance heuristic. Compare its execution time and performance with Dijkstra's algorithm on larger graphs.	Ishita Bijalwan
Comprehensive Integration Testing	Test the complete application workflow, including place search, category filtering, itinerary creation, route optimization, saving trips, and retrieving saved itineraries.	All Members
API and Backend Validation	Verify that the frontend communicates correctly with the C++ backend and that all API endpoints return the expected responses. Resolve any remaining issues related to routing, database access, or server configuration.	All Members
Error Handling and Reliability	Test invalid place IDs, empty itineraries, unreachable destinations, invalid requests, and database errors. Ensure the application handles these cases correctly.	All Members
Performance and Algorithm Validation	Test route optimization on larger datasets and validate the correctness of the generated paths, total distances, and travel times.	Ishita Bijalwan
Final Documentation and Demonstration	Update the README, document the final architecture and execution steps, organize the source code, and prepare a demonstration of the integrated application.	All Members

Project Outcome/Deliverables

The main outcome is a working web application that helps a person plan a trip on a map. A user will be able to type a place name and get suggestions from the Trie based autocomplete, add places to an itinerary, and ask for a route. The route is calculated by our own C++ graph code and shown on the Leaflet map. Saved itineraries stay available after the application is closed, because they are stored in SQLite.

The key deliverables are: the interactive map frontend, the search module with the Trie and hash table, the graph and routing module with the adjacency list, priority queue and Dijkstra, A* in the upcoming phase, the database and storage module with SQLite, vectors and linked lists, the complete source code in a shared repository, the final project report and the demonstration of the working system.

Progress Overview

The project has progressed from developing individual modules to integrating the frontend, routing algorithms, and database functionality.

The frontend uses HTML, CSS, JavaScript, and Leaflet.js to display an interactive map with place markers and location details. The search module uses a custom C++ Trie to provide prefix-based autocomplete, while the graph and routing modules use an adjacency-list representation and Dijkstra's algorithm to calculate shortest paths. A nearest-neighbour approach is implemented for multi-stop itinerary planning while this module remains partially complete. The application also includes SQLite database integration for retrieving places and categories and persisting itineraries. HTTP endpoints connect the application interface with the underlying application logic.

While we have integrated previously separate modules into a common application architecture and 85 to 90% of our project is completed, the remaining work focuses on completing A*, validating the end-to-end application, improving error handling, and preparing the final demonstration.

Codebase Information

Repository link: https://github.com/nandinirana05/Sangam-Your-City-Your-Experience/tree/main

Branch: The main branch holds the stable code, and there is one more branch where members work on their own modules before merging. The repository has folders for each part: Graph_construction, Place_Data_Structuring_Module, Route_and_itinerary_optimization_, SQLite_Database and webpage.

Important commits: initial project structure and folder setup, Leaflet map frontend and place markers, Trie and hash table for search and autocomplete, graph construction with adjacency list Dijkstra with priority queue and itinerary ordering, SQLite schema and persistence code, vectors and linked lists for the place data module.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Trie prefix search	Pass	Tested with sample place names and different prefixes, including a prefix that matches nothing.
Hash table lookup	Pass	Existing and missing place names return the expected result.
Graph construction and Dijkstra	Pass	Shortest path checked by hand on a small sample graph of places.
Map frontend (manual test)	Pass	Map loads, markers and popups work, and the page was checked on desktop and mobile screen sizes.
SQLite create, insert and read	Pass	Tables are created, places are saved and loaded back correctly.
Vector and linked list storage	Pass	Places are grouped by category and stop order is kept correctly.
Frontend and backend integration	Pending	Cannot be tested until the API endpoints are finished.

Deliverables Progress

Deliverable	Current Status	Progress / Remarks
Interactive Map Frontend	Completed	The Leaflet.js interface displays places on an interactive map. Frontend code retrieves place data through API calls.
Place Search and Autocomplete	Completed	A custom C++ Trie supports prefix-based search, and the frontend calls the search endpoint to retrieve matching places.
Graph Construction	Completed	The graph is centralized in `src/core/Graph.h` and constructed using place and connection data retrieved from SQLite.
Route and Itinerary Optimization	Completed — Core Functionality	Dijkstra's algorithm calculates shortest paths, while a nearest-neighbour approach orders multiple stops. Further testing and validation remain necessary.
A* Algorithm	Partially Completed	A basic framework is reported in the integration-status document, but heuristic implementation and performance benchmarking remain pending.
Database Schema and Persistence	Completed — Core Functionality	SQLite integration supports retrieving places and categories and saving and loading itineraries.
Place and Itinerary Storage	Completed	The backend includes itinerary persistence endpoints, while the database module stores itinerary information and ordered stops.
Category Grouping and Itinerary Management	Completed — Implementation Status	The integration-status document reports that category filtering and itinerary management are integrated with the application interface.
Backend and Frontend Integration	Implemented — Requires Final Validation	API endpoints and frontend calls are present. The complete application workflow should be tested before declaring integration fully validated.
Source Code Repository	In Progress	The repository contains the application modules, shared core components, database files, build scripts, and integration documentation. Final cleanup and documentation updates remain.
Final Project Report and Demonstration	Pending	Finalize the report, document the current architecture and implementation, and prepare the complete working demonstration.